Supervised Learning Report
CS7641: Machine Learning
Fall 2025

Michael Peluso
September 21, 2025
Professor Theodore J. LaGrow

***Abstract*—In this report, five supervised learning algorithms are applied to two separate datasets. The following models are reviewed: Decision trees, K-nearest neighbor, linear support vector machines, kernel support vector machines, neural networks. These models will be tested against hotel reservation data, containing ∼120,000 rows and a substantially larger US car accidents dataset that holds ∼1,000,000 rows. Both classification and regression models were tested, and metrics including F1, R2, and runtime are discussed extensively. Analysis such as sensitivity to noise and leakage, trade-off between speed and accuracy, and overall performance are derived from real data. This report aims to review and hypothesize about these models while grading them against each other, ultimately determining a single, reliable supervised learning model that best predicts the data.**

Introduction

Objective

The objective of this paper is to explore several supervised learning techniques and how they compare against different sets of data. Understanding each model and its underlying processes allows for further interpretation and analysis of the resulting metrics listed within this report. With knowledge of the internal mathematical concepts, hyperparameters can be tuned with precise intuitive interpretations. It offers a perspective into how the parameters affect the overall and how to further the model's accuracy. For example, understanding how decision trees depth affects overfitting can teach its engineer how to better tune the tree model. Finally, after tuning, fitting, and measuring the models, the results below aim to show in detail how these hyperparameters lead to higher scores with different models across two different datasets.

Hypotheses

- Decision Tree: Decision trees have a bias towards smaller, less complex structures. They tend to score better results with less simpler and more generalized decisions. Therefore, it is reasonable to conclude that a smaller maximum tree depth (`max_depth`) would restrict overfitting to highly specific decisions within the noise.
- K-Nearest Neighbor: Given the two datasets are noticeably large, it can be assumed that they contain plenty of noise. Forcing the k-nearest neighbor model to select fewer neighbors should reduce variance and balance bias within the data, ultimately helping to further generalize high-dimensional data.
- Kernel SVM: When splitting datapoints, the value C determines the marginal distance between classifiers and those datapoints. A higher C value creates more tightly fit decision boundaries. This should lead to more accurate margins and better classifications at the cost of higher runtimes and possibilities of overfitting.
- Linear SVM: Given that alpha for linear SVM models is the reciprocal of C in kernel models, a lower alpha value should result in less overfitting as mentioned for the Kernel SVM.
- Neural Network: Reaching convergence on large datasets require longer execution times, but increasing the learning rate (`learning_rate_init`) will encourage and enhance convergence on complex patterns by increasing weight adjustments. This should help guide the neural networks out of local minimums without requiring momentum.

Methodology

Overview

This system was built with a focus on modularity, scalability, and ease of use. It was designed to abstract low level logic behind higher level function calls, allowing development to scale without constant rearchitecting. The system is split into steps that begins the initialization of a main class and determines what model is used on what dataset and how its executed. Then the user-specified data is read, cleaned, and split, and passed through a machine learning pipeline automated by scikit-learn. It converts numeric and categorical features into a form acceptable for the given model. The new pipeline is tuned with predetermined feature values. Lastly, the models performance is measured, logged, and plotted for visual interpretations. Scikit-learn is used substantially throughout this project, as its model implementations are simple and efficient [1].

Initialization

In `main.py` a new initialization of `ModelingExperiment` is made per model test. The user can specify the dataset, target feature, supervised learning method, training model, subsample fraction, and specified random seed, whether to tune on model, custom

TABLE I
STAGES

Stage	Description
Initialization	Simply a portal for user to specify executions.
Data Processing	Loads, cleans, subsamples, and splits datasets.
Pipeline Construction	Builds preprocessing pipeline with chosen model.
Hyperparameter Tuning	Tunes model with grid/random search.
Model Analysis	Generates learning/complexity curves.
Evaluation	Logs performance metrics and plots.

hyperparameters (if tuning=False), cross-validation splits, feature combination cap.

Data Processing

The data was initially read from CSV file made available and downloaded from the Supervised Learning Report Guidelines [2]. The Hotels dataset needed to be cleaned by removing the suggested rows for the given target feature. This included `reservation_status` and `reservation_status_date`, as these introduce leaking due to their inherent relation to whether a reservation is cancelled. The target feature quite literally be derived from `reservation_status`. Because of the smaller size of the hotels dataset, further dataset-specific cleaning was not necessary. The later dataset required further cleaning, as millions of rows led to extensive run times. Again, the suggested fields, `End_Time` and `Weather_Timestamp`, were dropped because of their inherent leakage. `ID` and `Description` were removed as having fully unique values would not aid in any generalizing prediction. `Turning_Loop` and `Country`, each only have a single value, grouping all the data into a single category, setting their entropy to zero. `Source` was dropped because it split the data in half, and, unless the data occurred bias from either source, this should have no effect on training. `Bump`, `Roundabout`, and `Traffic_Calming` were removed for splitting less than one percent of the total rows into a second category. Additionally, `Zipcode` was truncated to five digits to tighten its memory footprint. In hoped of offering stronger signals, `Start_Time` was converted to a pandas datetime type before split into days, months, and boolean values (i.e. `Is_Weekday`) and being dropped. Finally, `Duration_Seconds` was derived from `Start_Time` − `End_Time` before dropping `End_Time` to minimize leakage. From this point on, both datasets are treated as identical. All numeric values are downcasted for memory efficiency. After cleaning, the hotel dataset row count was 87,138 with 69,710 test samples and 17,428 train samples. This left only 4.84 MB required to encode the data. Different sized subsets of US Accidents were used for time complexity issues, but a subset of one million rows has roughly 70% - 80% memory reduction. Both datasets use a 20% / 80% split to determine the sizes of the training and test sets respectively. hotel features {`country`, `agent`, `company`} and accident features {`Street`, `County`, `City`, `Zipcode`, `Airport_Code`} were encoded as high categorical features, while features with medium to low category counts were encoded as low categorical features. All other features were categorized as numerical features, including integers, floating point numbers, and boolean values. It is interesting that the hotels dataset has a target split of about 73% to 27%. An imbalanced target attribute highlighted possible challenges in binary classification. ROC-AUC is used to determine model discrimination, so it is important for a dataset with unbalanced splits. Also, the complexity matrices clearly showed the models' trouble classifying true "`is_canceled`" negatives, with a high amount of false positives across all models. Much of the focus during development was on the hotels dataset. The dataset's manageable size allowed for quick and iterative testing and tuning, shifting priorities to fleshing out the entire training system. The linear support vector machine (SVM) and decision tree trained quickly, allowing for many parameter combinations for testing. Deriving `Duration_Seconds` opened a door into practical uses in determining traffic patterns and clean up delays. With such a large span of features, including plenty of numerical values as well as high cardinality features, this set brought difficulties with noise and possible leakages. Much of the leakage was removed in cleaning, but models like SVMs and neural networks may have trouble seeing through the noise. On the other hand, the high row count restricted the number of features to test and cross validate per execution. A remediation for the time complexity issue was to lower the number of folds or total possible feature combinations each fold experienced.

Pipeline Construction

The scikit-learn `Pipeline` was used for an efficient workflow [1], [3] and was populated with a few data processing components to transform the column data for model tuning. The pipeline began with handling missing values. All numeric values use `SimpleImputer` set to "constant" to fill any missing values with the median of the set and avoid rare values that add noise. Columns with low to medium cardinality had missing values replaced with the median, or the most frequently seen value. Due to the model's high volatility and large example count, this was less sensitive than mean. Missing values in columns with high cardinality are left as zero, as predicting its true value may cause undesirable noise if calculated wrong. The next step in the pipeline was separating features into three main categories: numeral, low cardinality, and high cardinality columns. Separating values into these categories substantially increased computational speed via limiting dimensionality. With such a large number of rows and features, adding or removing a single feature heavily affects time complexity. This technique accounts for the curse of dimensionality, therefore limiting features with many different values from splitting into hundreds or thousands of new columns. High cardinality columns also use `CountEncoder` to replace the name of the group with their feature counts to mitigate dimensionality. This also aggregates rare groups for a better

TABLE II
DATASET SUMMARY

Dataset	Task	Rows	Train/Test	Features	Memory (MB)
Hotels	Classification	87,138	69,710 / 17,428	3/10/2	4.84
Accidents	Regression	997,408	797,926 / 199,482	12/13/5	124.50

generalization. All rows regardless of their value are then scaled using the `StandardScaler` to normalize the values between 0 and 1. This tightening of values scales the small values and minimizes outliers. These changes create a more stable dataset that is more robust against diverse data and outliers. The previous components were transformers that encode the data into an optimized form for the model fitting, tuning, and predicting. The end to the pipeline is the encoder – the imported model from scikit-learn that processes the data [4]. The following models were imported and used for calculating the results for this paper: `DecisionTreeClassifier/Regressor` [5], `KNeighborsClassifier/Regressor` [6], `SGDClassifier/Regressor` (Linear SVM) [7], `SVC/SVR` (Kernel SVM) [8], and `MLPClassifier/Regressor` (Neural Networks) [9]. These models were chosen due to compatibilities with the other modules and the addition to the tuning parameters they offered compared to their alternatives. Some classes lacked required attributes or had no reasonable way to train preferred settings.

Cross Validation

Cross validation was fundamental both the development and testing of these models. Regression models used `KFold` to split test sets into test and validation grids to ensure random partitioning for continuous targets. Classification methods, due to their categorical nature, were stratified using `StratifiedKFold` so that category percentages are preserved in each set. Given the severe `is_canceled` target imbalance, this methodology proved to keep class proportions accurate. Overall, implementing folds helped to mitigate bias from imbalanced target feature proportions, especially when the number of feature combinations were limited. GridSearch was initially implemented as an exhaustive search that trained on all possible combinations of tuning parameters. However, this was infeasible due to the rapid scaling of just a few parameters. Therefore, `RandomizedSearchCV` was implemented to limit combinations at a given threshold. After calculating every possible combination of parameters to test, few would be selected at random, fitted with cross validation of specified fold counts, and compared with previous fits. Randomized search preserved exploration with computational feasibility. The limiting threshold and fold counts are both specified by `ModelingExperiment`. Hyperparameters were also manually limited with specified possible values in `tuning_config.py`. Methodologies for deciding hyperparameters and their values is as follows.

Method-specific hyperparameter tuning

In order to efficiently tune many parameters in a single execution, they were listed in arrays in `tuning_config.py` and dynamically fit against their respective model. Most models were set with default values contained in the ranges that offered the least model prediction restriction. For example, `max_depth` was set high and `min_samples_leaf` was set low. This section lists the specific parameters and ranges used for calculated parameter combination. All models that allowed were set with `n_jobs` as -1 to take advantage of multiprocessing advantages.

Decision tree: To address the uneven class distribution in the hotels dataset, the class weight was set as imbalanced. The Criterion was set to entropy due to its prevalence in this course and sensitivity to impurity that better handles imbalanced data. The maximum tree depth was tuned to limit overfitting by controlling the depth of low and high complex trees. The bias towards smaller, simpler trees was evaluated here. The minimum leaves and required split samples were tested to for smoother trees and variance reductions to avoid learning the noise. Limited the maximum features introduced a bit of randomness in terms of how the tree was built and calculated. This offered a better generalization. Alpha was important in determining the amount of post-pruning, which helped mitigate overfitting. Due to the decision tree's fast computational speed, testing many parameters combinations was feasible. Overall, entropy and balanced weights aided the target feature imbalance in the hotels dataset, while pruning and depth limits prevented the model from overfitting to the accidents' noise.

K-Nearest Neighbors: Euclidean distance was selected as a standard L2 distance to address the many numeric features prominent in both sets. A brute algorithm ensured exact computation over the subsampled accident set. The number of neighbors to iterate over varied between datasets to avoid for long runtimes. But neighbor count directly determine sensitivity to noise, variance calculation, and overfitting. The ranges used offered plenty of options when tuning and explore the effects of varying locality.

Linear SVM: Tolerance and the initial learning rate were set to 0.01 to allow for early stopping to prevent training errors. Ranging alpha adjusted the regularization strength toward finding a boundary, while the penalty parameter varied sparsity. These offered a differing volatility sensitivity to change, with small alpha values fitting cleaner data and larger values preventing overfitting. Capping the maximum iterations encouraged convergence and ensured stability given the scale of the datasets. Using different learning methods increased

randomness and generalization, adapting to difference scales and data distributions. For the linear support vector machine, the tolerance and the initial learning rate ensured feasible execution times and prevented overfitting to noise, while the alpha and penalty ranges explored differing sparsity. Iterations were limited to address convergence and time complexity issues.

Kernel SVM: The kernel was set to "rbf" as per the assignment guidelines for non-linear patterns. Because this model was the most time consuming of those tested, turning on shrinking optimized training speeds. Altering C controlled regularization, where higher values provided a tighter fit, but found more noise. The kernel coefficient, gamma, was adjusted to fit feature variance. These values particularly vary the model's sensitivity to underfitting and overfitting data.

Neural Networks: For tuning the neural networks, the solver type was set to "sgd" for epoch customization. SGD with an adaptive learning rate led to more stabilized convergence. The hidden layers parameter took both (512, 512) and (256, 256, 128, 128) to compare deep vs shallow networks using a common batch size of 1024. All tests were conducted with `hidden_layer_sizes` set to either tuple for accurate comparisons. In order to comply with building the scikit-learn pipeline, `early_stopping` was set to false and was controlled via setting max iterations. `n_iter_no_change` was set to 2 as listed in the guidelines to trigger early stopping and balance training time. The learning rate was set to adaptive as suggested in scikit-learn's documentation for more optimized convergence by preventing model stagnation. This affected sensitivity to overfitting with lower values by executing shorter iterations and longer values fitting lighter data patterns. Tuning for several values of alpha allowed for customized weight adjustment strength to preserve the tradeoff between fitting feature signal and volatility. Different activation functions offered non-linearity diversity and added to randomness like Decision Tree's max features. ReLU used simpler computation while tanh and logistic activations tested for potential gradient stability.

Reproducibility

A random seed of 1 was set as default for all models, but can be adjusted within the `ModelingExperiment` parameters. This ensured the same randomization across executions.

Model Metrics

After the model is fit to its best parameters, a few more model predictions are made. First each model is measured on time to execute and computational resources each requires. Then each is modeled on a complexity curve against one parameter specified in the tuning configuration file. This complexity curve uses scikit-learn's `validation_curve` function [1] to display model performance between the changing hyperparameter values. Then the learning curve is computed and graphed using the same package's `learning_curve` function [1]. Metrics like F1 and R2 are measured and saved. Decision trees are drawn and modeled onto a pruning path plot, and neural networks are modeled with a custom function to display loss vs epoch. Finally, all graphs are plotted, and a full execution report is printed, listing all metrics, time and resource statistics, and model-specific hyperparameters.

Experiments and Results

Tuning

Classification models compared accuracy, weighted F1, average precision, and ROC-AUC, while regression models compared mean absolute error, mean squared error, and R-squared values. The default parameters were either given in the guidelines or specified to reduce execution time. The following are the hyperparameters tested for each model. The following lists each supervised learning model alongside and how it performed during tuning and fitting. The best parameters list each model's tuned hyperparameters that resulted in the highest learning scores. The metrics are reported using the final held-out test set evaluations, with the resulting hyperparameters after tuning via cross-validation.

Per-Algorithm Results

Decision Tree: Using the entirety of the hotels dataset of 87,138 rows, the model achieved strong results with an F1 score of 0.7420 and ROC-AUC of 0.8061. The tree had 126 leaves and was 17 layers deep. Its visually clumped layout signals strong signals to few features, possibly involving leakage. The learning curve shows F1 rising steadily until around 0.74 by 40,000 samples before plateauing, meaning there was high initial bias. Though the small gap between training and validation lines after a max depth of 10 indicates low overfitting [**?**], most likely to be due to pruning via ccp_alpha and depth limits. This supports the hypothesis that simpler trees lead to less overfitting. The moderate spread across features shows little reliance on a single feature, offering generalized, holistic results. Scoring a high ROC-AUC signals leakage, most likely from booking lead time or deposit type. Higher ROC-AUC is possible while still resulting in many false positives [10], aligning with a one-sided confusion matrix, favoring TN=11,148, over TP=2,001. The ROC curve shows a high area under the curve at 0.81 versus 0.5 at random, but the PR curve's slow drop-off suggests challenges with imbalanced classes. A low alpha in the pruning path shows that while unpruned trees avoid overfitting, they can be skewed with leaked features rather than generalizing [**?**]. The calibration curve showed accurate results at 0.5, but a visual dip with more positives, which signals overestimating subtle patterns [10]. For Accidents, testing 965,860 rows, the model produced 22 depth and 2,850 leaves. It achieved the highest R^2 of any model, 0.3342, but may be artificially bolstered due to a clear reliance on "`Start_Lng`", indicating heavy leakage. This suggests that the dataset was not accurately cleaned, or an encoding error with "`Start_Lng`". The tree contains obviously more spread out branches, displaying a well-generalized architecture. The learning curve indicates that there was minor overfitting that decreased with scaling data, given by the model stabilizing at around 0.35 versus 0.33

TABLE III
MODEL RESULTS - HOTELS DATASET

Model	Subsample	Runtime & Memory	AP	ROC	Acc	F1	CM	Best Parameters
DT	1.0	67.4s, 214.7MB	0.57	0.81	0.75	0.74	[[11148,1526],[2753,2001]]	min_samples_split: 400 min_samples_leaf: 100 max_features: 0.5 max_depth: None ccp_alpha: 0.0001
KNN	1.0	178.4s, 240.7MB	0.53	0.79	0.76	0.75	[[10817,1857],[2383,2371]]	n_neighbors: 5
SVM	1.0	2053.1s, 229.7MB	-	-	0.76	0.75	[[11410,1264],[2844,1910]]	C: 8 gamma: auto
NN	1.0	2060.9s, 172.8MB	0.59	0.82	0.76	0.74	[[11497,1177],[2993,1761]]	max_iter: 300 learning_rate_init: 0.01 hidden_layer_sizes: 512 alpha: 0.01 activation: relu
LSVM	1.0	74.0s, 226.5MB	-	-	0.73	0.66	[[12314,360],[4260,494]]	tol: 0.001 penalty: None max_iter: 5000 learning_rate: optimal alpha: 1e-05

TABLE IV
MODEL RESULTS - ACCIDENTS DATASET

Model	Subsample	Runtime & Memory	MAE	MSE	R2	Best Parameters
DT	1.0	516.8s, 316.4MB	0.46	0.39	0.33	min_samples_split: 200 min_samples_leaf: 200 max_features: 0.5 max_depth: 22 ccp_alpha: 0.0
KNN	0.3	1308.8s, 353.1MB	0.58	0.54	0.08	n_neighbors: 21
SVM	0.1	3486.2s, 204.5MB	0.57	0.57	0.02	C: 0.5 gamma: scale
NN	0.8	12065.6s, 230.3MB	0.49	0.41	0.28	max_iter: 300 learning_rate_init: 0.005 hidden_layer_sizes: 512 alpha: 0.0001 activation: relu
LSVM	1.0	482.9s, 406.8MB	0.60	0.55	0.06	tol: 0.0001 penalty: None max_iter: 20000 learning_rate: adaptive alpha: 1e-05

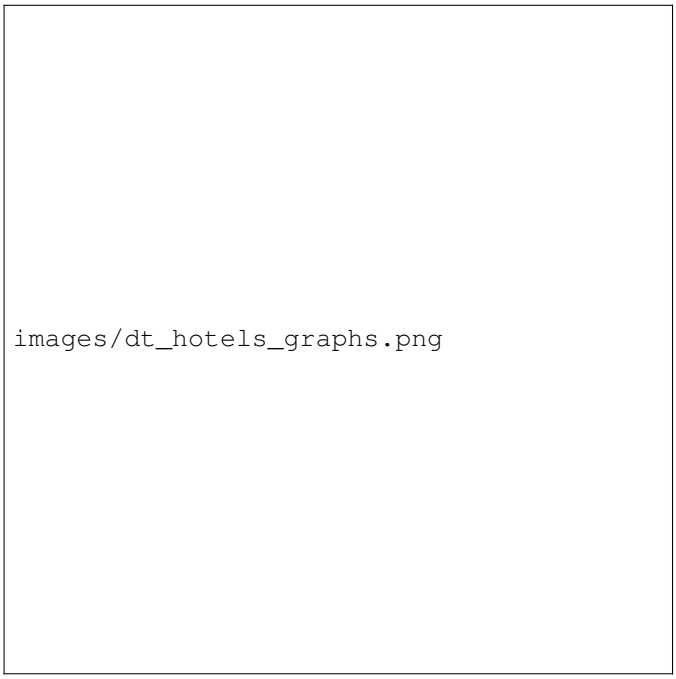

Fig. 1. All Graphs for Decision Tree (Hotels Dataset)

for the validation set [**?**]. The decision tree on the accidents dataset proves to have a much more stable difference between pruning parameters, validating the simplification hypothesis. Heteroscedasticity shown after 0.9 in the residual plot hints towards pre-processing errors. The data on both decision trees

Fig. 2. All Graphs for Decision Tree (Hotels Dataset)

shows positive effects of pruning by simplifying its architecture for more generalization, while having enough layers to accurately decipher variance. While these tests showed that decision trees are sensitive to imbalanced classes, they predict more accurately with smaller, simpler datasets than larger datasets that risk capturing noise, as shown by the F1 and ROC-AUC scores, lower runtime, and effective pruning.

K-Nearest Neighbor: With an optimal k=5, the KNN model achieved F1 of 0.7521 and ROC-AUC of 0.7865, scoring a bit above baseline accuracy but handling imbalance moderately well. F1 is roughly stable at $\sim$0.83 while validation rises more noticeably from 0.73 to 0.75, indicating high levels of overfitting and variance in high-dimensional features [**?**]. This defies the assumption that a low neighbor count, k=5, minimizes overfitting. The complexity curve exemplifies the curse of dimensionality present in the validation set peaking at 5 neighbors and slightly declining afterwards, losing local precision. The training data shows a steep decline from k=1 at $\sim$0.95 and plateaus with k=11 at $\sim$0.85, showing significant overfitting ultimately solved with increasing k values. The ROC and PR curves, as well as the confusion matrix, shows similar imbalance challenges as earlier, faring even worse than the decision tree. This is most likely due to K-NN's local voting sensitivity that exacerbates class bias majorities. The

Fig. 3. All Graphs for K-Nearest Neighbor (Hotels Dataset)

model performed much worse on the accidents dataset, scoring low metrics of R^2=0.0794, MAE=0.5784, and MSE=0.5352. This reflects K-NN's poor predictive power with scaled data and stronger noise [**?**]. As in the hotels dataset, the model dangerously overfits when k is small, measuring R2 for the training set at 1.00 with the validation set scoring -0.7. But the difference closes drastically with increasing k values, leveling off after k=11, showing that increasing the neighbor count minimizes variance by capturing more points and generalizing their respective values. The learning curve measured very similar to that of the hotels test, with slightly less leakage given the training set's steeper slope. Overall, the model

Fig. 4. All Graphs for K-Nearest Neighbor (Accidents Dataset)

tends to show difficulty overcoming larger, noisier and higher-dimensional data reflected in a low R2 values compared to a moderate F1 score. K-NN appears to score better with simpler data despite the intensive feature handling performed prior. While the hypothesis incorrectly suggested that larger neighborhoods would better handle high-dimensional, it aligns with larger k values resulting in less overfitting.

Kernel Support Vector Machine: The complexity plot for hotels shows increasing F1 scores as C increases to 10, with no visible slow down. Therefore, stronger regularization appears to enhance performance, though no optimal range of C can be interpreted. The confusion matrix displays classification that is balanced yet imperfect, with 11,410 true negatives but 2,844 false negatives and 1,264 false positives. The learning curve shows a gradual decreasing F1 training score from 0.78 to 0.75 with a moderately fluctuating validation scores stabilizing just below 0.74. The gap at smaller training sizes, persisting into larger training sizes suggests medium levels of variance, converging as training sizes increase, pointing to overfitting and leakage [**?**]. A clear dip in training R2 from

Fig. 5. All Graphs for Kernel SVM (Hotels Dataset)

0.2 to 0.1 and validation R2 from ∼0.75 to ∼0.66 indicated C=2 best tunes this model for the highest results. Validation R2 decreasing over all values suggests less regularization at the cost of overfitting. The difference in R2 values further proves that overfitting has taken place. The learning curve shows the training R2 score decreasing from 0.22 to 0.15 while validation R2 rises modestly from 0.05 to 0.075. Therefore, the model has high levels of bias but has potential to improve with more data. The residual plots spread between -3 and 3 across ranges 7.5 to 9.5, violates homoscedasticity and underscores model misspecification, compounded with low MSE and MAE values, hints at poor feature engineering prior to testing. The consistent results between the two datasets show kernel

Fig. 6. All Graphs for Kernel SVM (Accidents Dataset)

support vector machines to be sensitive to leakage and struggle with non-diagnostic features [10]. As correctly stated in the hypothesis, the complexity curve displays a higher R2 score as C increases to a point at the cost of substantial overfitting. Though the kernel SVM appears to handle class imbalances far greater than methods previously mentioned.

Neural Networks: The calibration curve for the hotels dataset significantly deviates from a perfect deviation, with overconfidence at higher probabilities that most likely result from miscalibration from possibly leaked features [**?**]. The model complexity curve shows tightly coupled F1 values for both training and validation sets, proving well amid noise. Their continuous increasing scores with increasing learning rates indicate that higher complexity captures more variance while risking incorporating leakage [10]. The confusion matrix demonstrates the model's strong true negative detection (12,186) but poor positive recall with only 844 true positives versus 3,910 false negatives. It scored one of the worst splits, faring poorly with imbalanced data. The learning curve's clear convergence and increasing F1 scores point to a reduced variance but persistent bias from non-predictive features. The ROC and PR curves show a sharp precision drop and indicate moderate discrimination, emphasizing any present leakage. The learning curve in the accidents set shows moderately

Fig. 7. All Graphs for Neural Network (Hotels Dataset)

stable training scores up to about 15,000 examples, while the validation score quickly increases. This shows an apparent bias that is mediated with more data. Gaps between the scores show a lack of convergence and possible masking of generalization via leakage. Like earlier models, the scattered residuals coupled with metrics like MAE 0.4897, MSE 0.4147, and R2 0.2809 reveal a limited explanatory power. These

Fig. 8. All Graphs for Neural Network (Accidents Dataset)

models seem to withstand noise better than previous models, but show a large vulnerability to appearances of leakage in high-dimensional feature spaces of 28 and 39 features respectively. The metrics between deep, 4-layer and shallow, 2-layer networks posed very little difference other than noticeably more accurate calibration curve. The deep network (256, 256, 128, 128) seems model the more complicated patterns better than its shallower counterpart. More layers lead to learning more complex learning and can better map to feature signals. A dataset like hotels has more than enough complex features to train on.

Linear Support Vector Machine: The linear SVM, while easily computable, posed severe problems during testing, including incapabilities with the overall workflow, and constant errors when attempting to collect data. Despite each model using the same processed data, this seems to have led to poor testing metrics. The model tested on hotels shows rapid convergence of both training and validation sets with weighted F1 scores stabilize to about 0.635 with decreasing values of alpha around 1e-05. This shows that there is sensitivity to regularization changes for executions with higher alpha values. The confusion matrix with poor positive class predictions emphasizes a leakage in the data preprocessing [10], scoring high accuracy but a lower F1 score. It predicted a staggering 12,314 true negatives with only 494 true positives against 4,260 false negatives. The volatile results in the learning curve show severely unpredictable increasing and decreasing trends, pointing towards high variance and insufficient data to mitigate the noise. This can lead to exacerbated overfitting in larger datasets [**?**]. With a significantly low R2 value of 0.0587,

Fig. 9. All Graphs for Linear SVM (Hotels Dataset)

the flat complexity curve shows that this model was very insensitive to regularization and insufficient in capturing any meaningful variance. Feature preservation was not correctly applied to allow this model to accurately predict [10]. The learning curve staying constant at zero with a dramatic dip to -1.55 and soon back to zero displays a clear bias and potential leakage, resulting in the model's inability to generalize and converge. The scatter plot shows clustered data, with residuals widening from 8 to 13, indicating heteroscedasticity and, therefore, suggesting more possibilities of leakage. Combined with metrics like MAE of 0.5951 and MSE of 0.5473, a clear amount of leakage occurred in this model's predictions. Because the program set a constant random seed, each model

Fig. 10. All Graphs for Linear SVM (Accidents Dataset)

was fit with the same exact preprocessed data. Given the clear findings of leakage within the linear SVM results, it can be assumed that these models are highly sensitive to leakage and struggle with data not extensively processed.

Cross-Algorithm Comparison

Overall, the decision tree algorithm proves to be the most efficient program with the fastest computation speed (∼15s

on Hotels and ∼45s on Accidents) versus prediction stability trade-off. Despite its difficulty with noisy datasets, it proved accurate with respect to the competing algorithms on both the hotels and accidents datasets (e.g., F1=0.7420/ROC-AUC=0.8061 on Hotels; R^2=0.3342/MAE=0.4606 on Accidents). Neural networks proved to capture complex patterns best among the models (e.g., F1=0.7405/ROC-AUC=0.8174 on Hotels; highest R^2=0.2809/MAE=0.4897 on Accidents), but lacked the efficiency of the decision trees, requiring about 900 and 1800 seconds respectively. K-nearest neighbors and kernel support vector machines performed well within the hotels dataset, but failed when scaling to the larger set, exemplifying the curse of dimensionality and high computational demands. The linear support vector machine shown to be the worst model of all, providing flat curves, proving to be unsuitable without heavy feature engineering (Hotels: F1=0.6604/ROC-AUC=0.70; Accidents: R^2=0.0587/MAE=0.5951).

CONCLUSION

This supervised learning exploration into the Hotels and US Accidents datasets with various machine learning algorithms reveal distinct strengths and weaknesses among each model, and provided practical lessons, tools, and further insights into implementing such techniques. Key steps include thorough data cleaning and feature engineering, model implementation and hyperparameter tuning, and output and metric analysis for further correction. Without critically engineered datasets, all models, regardless of their implementations, will fail to accurately predict new examples and succumb to the noise and leakages apparent in all data. When choosing the correct model for a specific use case, understanding its package requirements, hyperparameters, and time constraints are imperative for correct implementation. Lastly, machine learning is an iterative process. It requires constant tuning, adjustments, and analysis in order to accurately model abstract patterns. Though few of these models failed to model the patterns present in the provided datasets, their proceeding analyses resulted in far more learning.

REFERENCES

[1] scikit-learn developers, "scikit-learn: Machine learning in python," https://scikit-learn.org/stable/index.html, 2023, accessed: 2025-09-27.

[2] T. J. LaGrow, "Supervised learning report guidelines, cs7641: Machine learning, fall 2025," 2025, georgia Institute of Technology.

[3] AssemblyAI, "Scikit-learn pipeline tutorial with python code examples," https://www.youtube.com/watch?v=jzKSAeJpC6s, 2022, accessed: 2025-09-27.

[4] D. Science, "Understanding pipeline in machine learning with scikit-learn (sklearn pipeline)," 2021. [Online]. Available: https://www.youtube.com/watch?v=jzKSAeJpC6s

[5] scikit-learn developers, "Decision trees — scikit-learn documentation," https://scikit-learn.org/stable/modules/tree.html, 2024, accessed: 2025-09-27.

[6] ——, "Nearest neighbors — scikit-learn documentation," https://scikit-learn.org/stable/modules/neighbors.html, 2024, accessed: 2025-09-27.

[7] ——, "Stochastic gradient descent — scikit-learn documentation," https://scikit-learn.org/stable/modules/sgd.html, 2024, accessed: 2025-09-27.

[8] ——, "Support vector machines — scikit-learn documentation," https://scikit-learn.org/stable/modules/svm.html, 2024, accessed: 2025-09-27.

[9] ——, "Neural network models (supervised) — scikit-learn documentation," https://scikit-learn.org/stable/modules/neural$_n etworks_s upervised.html, 2024, accessed$: $2025 - 09 - 27$.

[10] J. Starcke, J. Spadafora, J. Spadafora, P. Spadafora, and M. Toma, "The effect of data leakage and feature selection on machine learning performance for early parkinson's disease detection," *Bioengineering*, vol. 12, no. 8, p. 845, 2025. [Online]. Available: https://www.mdpi.com/2306-5354/12/8/845